All Lecture Notes for SFWRENG 3DB3

Matthew Farah, ⟨farahm11@mcmaster.ca⟩, 400468251

October 23, 2025

Contents

Introduction

- A *sequential* system is a system composed of a single part which performs actions according to some defined order.
- A *processing element* is defined as an environment in which a process is run, which is capable of allocating resources such as memory to the process running on it.
  - By default, the word "process" refers to the execution of a sequential system.
- *Concurrent* and *parallel* systems are systems composed of independently operating parts functioning together to perform some task.
  - Concurrent and parallel systems are not the same thing, although they are often used interchangeably.
- Concurrent systems consist of processes which perform actions with logical simultaneity.
  - Two concurrently running processes interleave their actions on the same processing element.
- Parallel systems consist of processes which perform actions with physical simultaneity.
  - Two processes running in parallel require that each process run on a separate processing element.
- Concurrent systems can be described as the composition of two or more sequential processes.
- There exist many ways to model concurrent systems:
  1. *Finite State Processes* (FSPs)
     - Finite State Processes are an algebraic description of sequential and concurrent processes.
     - Also called *process algebras*.
  2. *Labelled Transition Systems* (LTSs)
     - LTSs are extensions of finite state machines used to visually represent sequential and concurrent processes.
     - *Labelled Transition System Analyzers* (LTSAs) are tools used to visualize, animate, and analyze the behaviour of concurrent systems.
  3. *Petrinets*
     - Petrinets are an alternate visualization of concurrent processes.

Finite State Processes

- FSPs are algebraic models used to represent the behaviour of sequential and concurrent processes.
  - FSPs are composed of `PROCESSES` and `actions`.
    - An `action` is a thing which the system does.
      - An `action` must be atomic (can only do one thing).
    - A `PROCESS` perform an action and calls another process.

Ex. Represent a process which performs action `x` and then calls another processes `P` as an FSP.

Solution:

$$(\texttt{x} \rightarrow \texttt{P})$$

- Repetitive behaviour is modelled by recursively calling processes.

Ex. Represent a simple on-off switch as an FSP.

Solution:

$$\begin{aligned} \texttt{SWITCH} &= \texttt{OFF} \\ \texttt{OFF} &= (\texttt{on} \rightarrow \texttt{OFF}) \\ \texttt{ON} &= (\texttt{off} \rightarrow \texttt{OFF}) \end{aligned}$$

- *Process substitution* is the act of replacing processes with their definitions to produce more succinct FSPs

Ex. Using process substitution, use the FSP for Example 2 to product an FSP for an on-off switch with only two processes.

Solution:

$$\texttt{SWITCH} = \texttt{OFF}$$
$$\texttt{OFF} = (\texttt{on} \rightarrow (\texttt{off} \rightarrow \texttt{OFF}))$$

- FSPs do not need to be deterministic.
- In FSP *choices* allow a process to execute processes non-deterministically.
  - Denoted as "`|`".

Ex. Represent a process which can perform action `x` then execute process `P` or perform `y` then execute `Q`.

Solution:

$$(\texttt{x} \rightarrow \texttt{P} \mid \texttt{y} \rightarrow \texttt{Q})$$

- A *boolean condition* is a statement which can either be true or false.
  - A boolean condition is said to have a *finite domain* if there exists a finite number of cases needed to be checked to evaluate it as true or false.
    - For example, the boolean condition `x = 0` has a finite domain, however `x > 0` does not.
- A *guarded action* is an action in a choice can only be executed so long as some boolean condition is met.
  - The condition allowing a guarded action to be taken is specified with the `when` keyword.
  - The boolean condition must have a finite domain.

Ex. Represent a process which can perform action `x` then `P` ONLY IF a condition `B` is true or perform `y` then execute `Q`.

Solution:

$$(\texttt{when B x} \rightarrow \texttt{P} \mid \texttt{y} \rightarrow \texttt{Q})$$

- The *alphabet* of a process is defined as the set of all possible actions that a process can perform.
  - The FSP representation of a representation implicitly defines the alphabet of a process.
  - If two FSPs have no actions in common, their alphabets are said to be *disjoint*.
  - An action is said to be *shared* between two processes if it exists in both of their alphabets.
- An *alphabet extension* is used to specify additional actions available to a process not defined in its FSP.
- If two processes are running concurrently, they are said to be running in *parallel composition*.
- The actions of a system composed of two concurrently running processes is an interleaving of the actions of each process.
  - The order of the actions taken by each system must be preserved.
    - For example, if a process `P` can execute action `a` then `b`, and process `Q` can execute `c`, a valid output of the system `(P||Q)` could be `a` $\rightarrow$ `c` $\rightarrow$ `b` but `b` $\rightarrow$ `a` $\rightarrow$ `c` would be invalid, since in `P`, `a` must be executed before `b`.
- If two processes running in parallel composition have shared actions, then the shared actions must be executed simultaneously by both processes.
- FSPs use the parallel composition operator to declare two processes as running concurrently.
  - Denoted as "`||`"
  - The parallel composition operator is both associative and commutative.
- Petrinets allow for much easier graphical representations of concurrent systems than LTSs.

Labelled Transition Systems

- LTSs are finite state machines used to graphically represent the FSP description of a process.
  - Starting states are denoted as a circle with a dot in the middle.

Ex. Represent the following FSP as an LTS.

$$\begin{aligned} \texttt{A} &= (\texttt{a} \rightarrow \texttt{B} \mid \texttt{b} \rightarrow \texttt{C}) \\ \texttt{B} &= (\texttt{a} \rightarrow \texttt{B} \mid \texttt{b} \rightarrow \texttt{D}) \\ \texttt{C} &= (\texttt{d} \rightarrow \texttt{C}) \\ \texttt{D} &= (\texttt{a} \rightarrow \texttt{C} \mid \texttt{c} \rightarrow \texttt{B}) \end{aligned}$$

Solution:

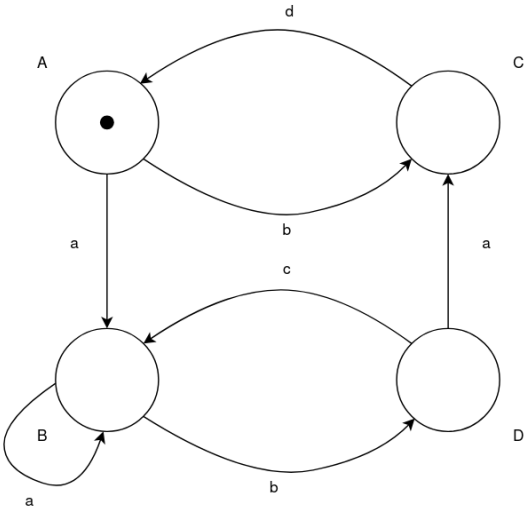

Figure 1: LTS representation of the given FSP.

- FSPs can nest processes within other processes (such as through processes substitution) to make FSPs more concise.

  - Directly translating an FSP with nested processes into an LTS will yield an incorrect representation of the FSP.

- An FSP is said to be put into *elementary/normal form* if all nested processes in the FSP are expanded out into separate processes.
  - FSPs must be put into elementary form to be correctly represented as an LTS.

Ex. Represent the following FSP as an LTS.

$$\begin{aligned}\mathtt{A} &= (\mathtt{a} \to \mathtt{b} \to \mathtt{B} \mid \mathtt{b} \to (\mathtt{a} \to \mathtt{c} \to \mathtt{A} \mid \mathtt{b} \to \mathtt{B}))\\ \mathtt{B} &= (\mathtt{a} \to \mathtt{c} \to (\mathtt{a} \to \mathtt{A} \mid \mathtt{b} \to \mathtt{B}))\end{aligned}$$

Solution:

First, we must put our FSP into normal form.

$$\begin{aligned}\mathtt{A} &= (\mathtt{a} \to \mathtt{B}_1 \mid \mathtt{b} \to \mathtt{D})\\ \mathtt{B} &= (\mathtt{a} \to \mathtt{A}_2)\\ \mathtt{D} &= (\mathtt{a} \to \mathtt{A}_1 \mid \mathtt{b} \to \mathtt{B})\\ \mathtt{A}_1 &= (\mathtt{c} \to \mathtt{A})\\ \mathtt{A}_2 &= (\mathtt{c} \to \mathtt{B}_2)\\ \mathtt{B}_1 &= (\mathtt{b} \to \mathtt{B})\\ \mathtt{B}_2 &= (\mathtt{a} \to \mathtt{A} \mid \mathtt{b} \to \mathtt{B})\end{aligned}$$

Now that our FSP is in elementary form, we can represent it as:

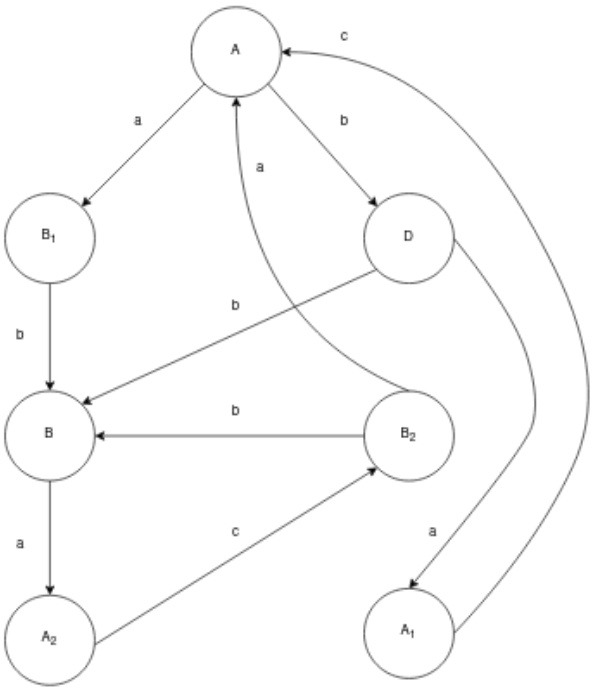

Figure 2: LTS representation of the given FSP.

Petrinets

- Petrinets are used to represent concurrent systems.
  - Petrinets are bipartite graphs.
  - An alternative representation to LTSs.
- There exist different kinds of Petrinets.
- An *elementary* Petrinet is composed of the following elements:
  1. *Places.*
     - Places represent a resource (essentially a state) that the system can be in.
     - Depicted by a circle.
  2. *Transitions.*
     - Transitions represent an action that a system performs.
     - Depicted by a rectangle.
  3. *Arcs.*
     - Arcs represent directed connections between places and transitions.
       - Since Petrinets are bipartitie, arcs cannot connect places to places or transitions to transitions.
     - Depicted by an arrow
  4. *Tokens.*
     - Tokens represent the current state of the system.
     - Tokens exist within places and move between places according to the arcs connecting places to transitions.
     - A petrinet can have multiple tokens simultaneously.
     - The current position of tokens in the system is depicted by a dot in the place the token resides in.
- Relative to a given transition:
  - An *input* place is a place which connects to that transition.
  - An *output* place is a place which that transition connects to.
- For a transition to fire, all input places to that transition must have a token inside it.
- Multiple transitions satisfying this condition can fire at this same time.
  - This is how Petrinets model the simultaneous execution of an action.
- When a transition fires, all output places are populated with a token and all input places are depopulated of their tokens.
- A given place can be both an input and output place of a transition.
- A *deadlock* describes a situation where tokens are arranged in such a way that there exists no valid transition for any tokens to take.
- The *firing sequence* of an elementary Petrinet are sequences of valid transitions that can be taken based on the initial location of tokens.

- The Petrinet representation of an LTS representing a concurrent system can be attained by representing the processes in parallel composition as Petrinets and "glueing" together shared transitions.
- *Labelled elementary Petrinets* are essentially the same as elementary Petrinets, with the disintion being that if two transitions can be fired simultaenously, they must have distinct labels.
- *Reachability graphs* are finite state machines used to represent the behaviour of Petrinets in LTS form.
  - For a given FSP, the reachability graph of the FSP's Petrinet representation is equivalent to the FSPs LTS representation.
  - Reachability graphs may allow for actions to be executed simultaneously.

Ex. Represent the following FSPs:

$$\begin{aligned} \texttt{MAKER} &= \texttt{make} \rightarrow \texttt{ready} \rightarrow \texttt{MAKER} \\ \texttt{USER} &= \texttt{ready} \rightarrow \texttt{user} \rightarrow \texttt{USER} \\ ||\texttt{MAKER\_USER} &= \texttt{MAKER} \,||\, \texttt{USER} \end{aligned}$$

(a) As LTSs.

Solution:

The LTSs for these FSPs are given by:

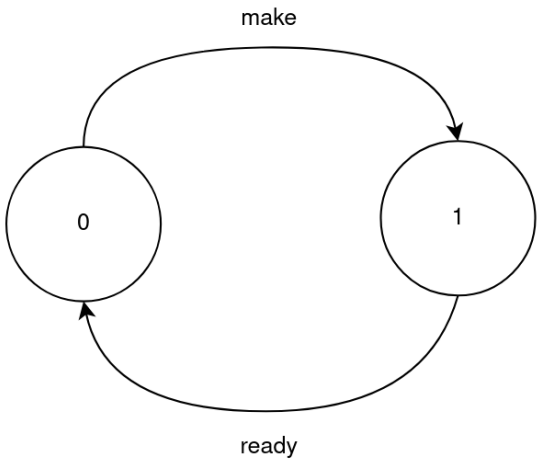

Figure 3: LTS representation of `MAKER`.

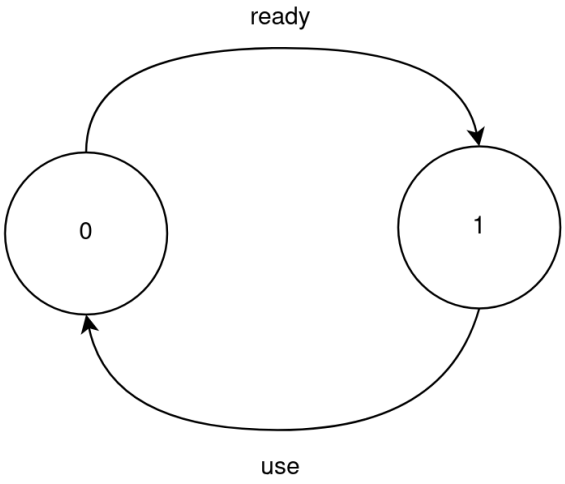

Figure 4: LTS representation of `USER`.

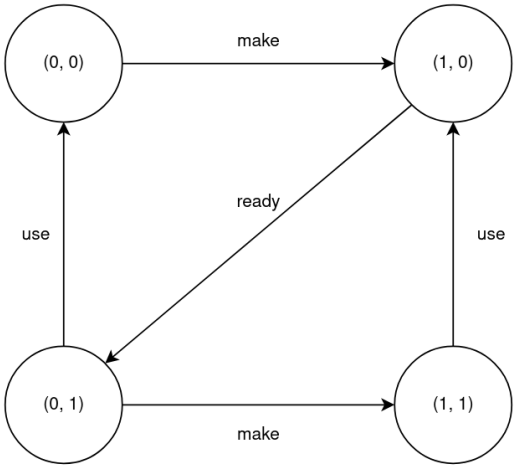

Figure 5: LTS representation of `MAKER_USER`.

(b) As Petrinets.

Solution:

The Petrinets for these FSPs are given by:

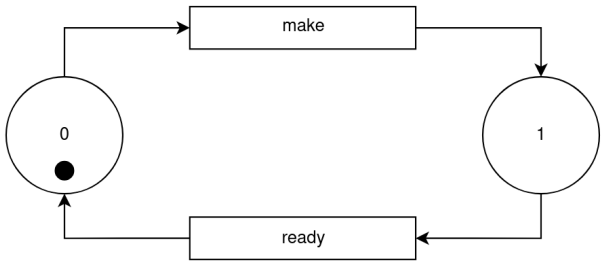

Figure 6: Petrinet representation of `MAKER`.

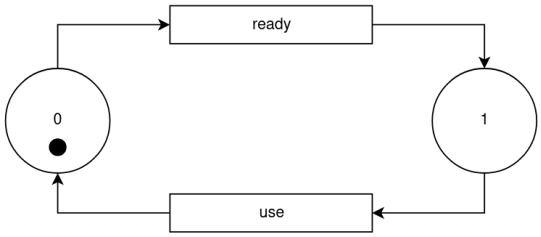

Figure 7: Petrinet representation of `USER`.

By glueing these two Petrinets by the common transition "`ready`", we can easily represent the Petrinet for `MAKER_USER` as:

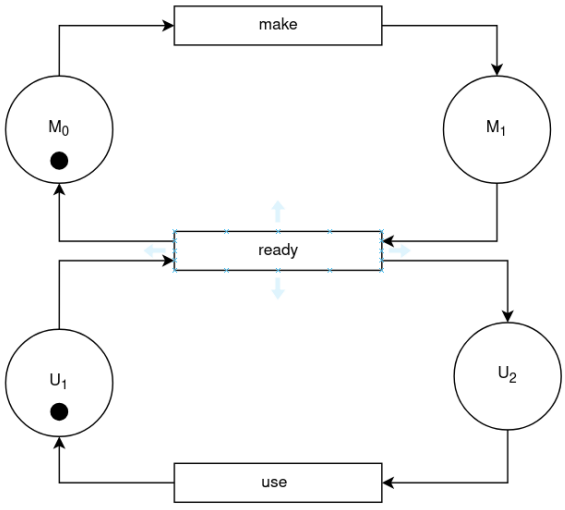

Figure 8: Petrinet representation of `MAKER_USER`.

Ex. Construct the reachability graph for the Petrinet representation of the `MAKER_USER`

FSP.

Solution:

The reachability graph for the Petrinet representation of `MAKER_USER` is given by:

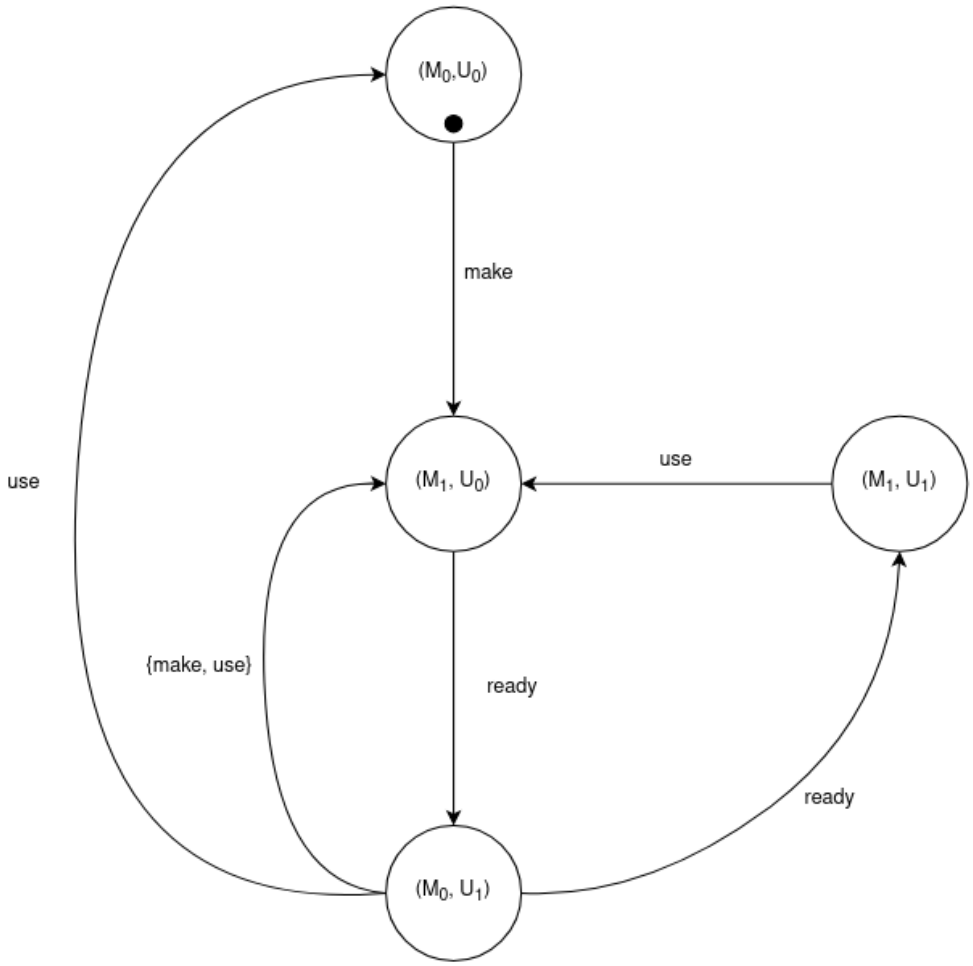

Figure 9: Reachability graph of `MAKER_USER` Petrinet.

Bisimilarity

- The *traces* of a process `P` to some state `p` in its LTS representation is the set of all sequences of actions which can be taken to reach `p`.
  - The traces of a process to its starting state is denoted by Trace(`P`).
- Two processes, `P`, `Q` are said to be *equivalent* if for some third process `R`, (`R` || `P`) = (`R` || `Q`).
  - `P` and `Q` are denoted as being equivalent by `P` ≡ `Q`.
- If `P` and `Q` are sequential or deterministic processes, then `P` ≡ `Q` $\iff$ Traces(`P`) = Traces(`Q`).
- However, in any other case, it is not sufficient for the traces of the two processes to be equal, since under parallel composition, two processes with the same traces can behave differently.
  - For example, for non-deterministic processes `P` and `Q` where Trace(`P`) = Trace(`Q`), it can be true that $(P \mid\mid R)$ can deadlock whereas $(Q \mid\mid R)$ does not for some process `R`.
- Hence, in the context of concurrent systems, equivalence is redefined as *bisimilarity*.
- Consider the LTS representations of `P` and `Q`:
  - Two states `p` ∈ `P` and `q` ∈ `Q` are said to be bisimilar if every action at `p` can be executed at `q`, and likewise if every action at `q` can be performed at `p`.
    - `p` and `q` being bisimilar is denoted `p` ≈ `q`.
    - Bisimilarity is both associative and commutative.
    - Important to note that a single state in `P` can be bisimilar to multiple states in `Q` and vice versa.
  - Two processes `P` and `Q` are said to be bisimilar if, for every state `p` ∈ `P` reachable by traces t, those some traces reach a state `q` ∈ `Q` which is bisimilar to `p`.
- `P` and `Q` are denoted as being bisimilar by `P` ≈ `Q`.
  - `P` ≈ `Q` $\implies$ (`P` || `R`) = (`P` || `R`) for all processes `R`.
- Bisimilarity cannot distinguish between two processes which can exhbit simultaneity or interleaving

Ex. Determine whether the following LTSs are bisimilar.

(a)

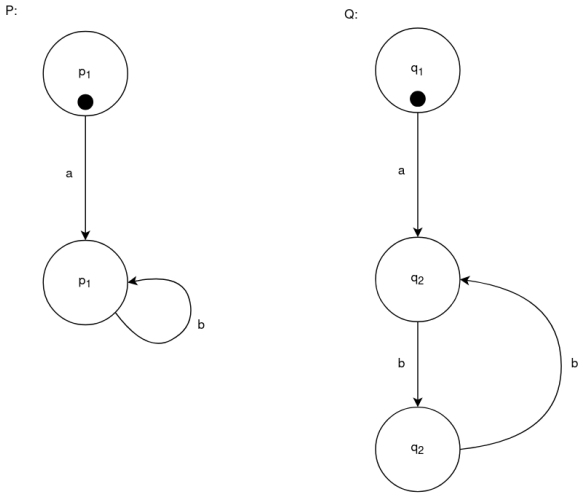

Figure 10: LTSs for part (a).

Solution:

They are bisimilar

(b)

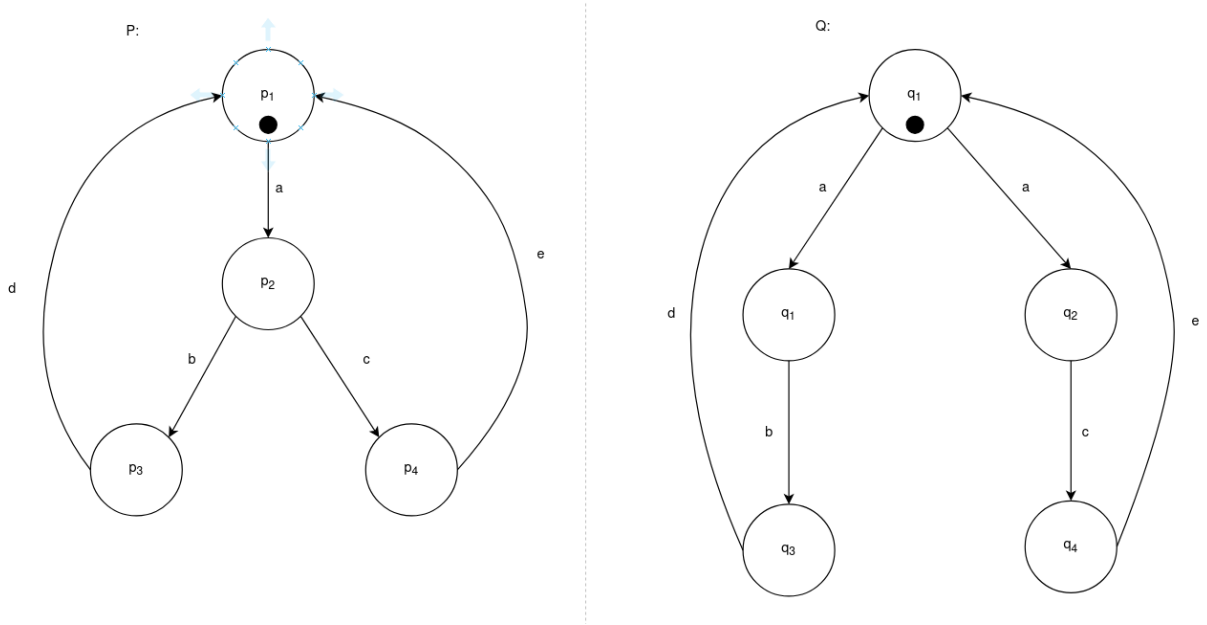

Figure 11: LTSs for part (b).

Solution:

They are not bisimilar

Actions and Labels

These notes are a bit of a stub because I don't totally understand what this lecture was talking about, I'll try to update with better notes in future.

- *Action relabelling* is the act of renaming actions in a process.
  - This is done to force concurrently running processes to execute certain actions simultaneously, even if they have different names.
- Actions can be relabelled with a prefix.
- *Action hiding* is the act of removing certain actions from the alphabet of a process.
  - This is done to prevent actions with the same name from being shared between two concurrently running processes.
  - Silent actions are relabelled as τ.
- *Structural diagrams* are used to represent:
  1. Concurrent systems.
  2. Action relabelling.
  3. Action hiding.

Mutual Exclusion and Locks

- *Updates* are actions which modify a stateful variable.
- *Intereference* occurs when updates are performed arbitrarily, potentially leading to unexpected behaviour and race conditions.
- *Mutual exclusion* is the process of preventing two concurrently running process from being able to modify data at the same time to prevent interference.
  - Mutual exclusion is implemented using *locks*.
    - When a process wants to be able to read/write to data, it must request a lock for that data. If no other process has a lock on the data, the lock is acquired by the process, giving it exclusive access to that data until the lock is released.
- An *active* process is a process which initiates actions.
- A *passive* process is a process which responds to actions.
- A *semaphore* is a variable used to implement synchronization between concurrently running processes.
- A process is *awakened* if **I DON'T FULLY UNDERSTAND WHAT AWAKENING IN THIS CONTEXT MEANS**
  - Semaphores can only take on non-negative integer values.
- Only the following operations can be performed on a given semaphore `s`.
  - `up(s)`
    - If a process is blocked on `s`, then the process is awakened, otherwise, `s` is awakened.
  - `down(s)`
    - If `s` > 0, the `s` is decremented, otherwise, the process calling `down(s)` is blocked.
- *Binary semaphores* are a special type of semaphore which can only take on the values of 0 or 1.
  - Binary semaphores are used to implement locks on data.